

Projeto 2 - Tópicos em Arquitetura e Hardware

Nome: Cleidiana Reis dos Santos

Matrícula: 298254

Introdução

A plataforma Litex é utilizada para a construção de SoCs (System-on-Chip) sobre FPGAs. FPGAs são dispositivos que podem ser reconfigurados para desempenhar diversas funções. O Litex simplifica a integração rápida de componentes essenciais como CPU, memória e periféricos. Este projeto teve como objetivo explorar a plataforma Litex por meio dos seguintes passos:

- Executar um exemplo completo da plataforma para compreender seu funcionamento.
- Desenvolver e executar um novo programa.
- Substituir o processador padrão por outro compatível para observar as diferenças.
- Adicionar e utilizar um novo periférico com a placa Tang Nano 9K.

Considerações Iniciais sobre o Projeto:

O projeto foi executado e testado considerando o seguinte ambiente de desenvolvimento:

- Sistema de instalação utilizado: Ubuntu 24.04.2 LTS
- Python 3.9.0

O código utilizado no projeto está documentado no github: <https://github.com/Cleidiana/Litex-TangNano9k>.

Para a execução correta dos comandos, é necessário seguir o padrão de pasta apresentado pela instalação Litex. Caso necessário, os caminhos dos arquivos devem ser ajustados.

Passo 1 – Execução de um Exemplo Completo da Plataforma

Ferramentas Necessárias:

Verilator: Instalado através do comando `apt-get install verilator`.

Execução:

A instalação do Litex seguiu o tutorial disponível no link oficial da Enjoy-Digital (<https://github.com/enjoy-digital/litex/wiki/Installation>). Primeiramente, foi feito o download do repositório oficial do Litex. Em seguida, foi concedida a permissão para execução do instalador. A instalação foi realizada com os seguintes comandos:

```
wget
https://raw.githubusercontent.com/enjoy-digital/litex/master/litex
_setup.py
chmod +x litex_setup.py
./litex_setup.py --init --install --user
```

O complemento da instalação da ferramenta Litex para utilização de CPU e execução de códigos em placa é feita da seguinte maneira:

```
pip3 install meson
./litex_setup.py --gcc=riscv
```

Após a instalação, pode-se criar a simulação de hardware com o seguinte comando:

```
litex_sim --cpu-type="tipo_cpu"
```

O comando `litex_sim --cpu-type=?` exibe as CPUs disponíveis para simulação. Como exemplo básico, será utilizada a VexRiscv, que pode ser simulada através do comando:

```
litex_sim --cpu-type=vexriscv
```

A tela da Figura 1 mostra o sucesso da simulação.

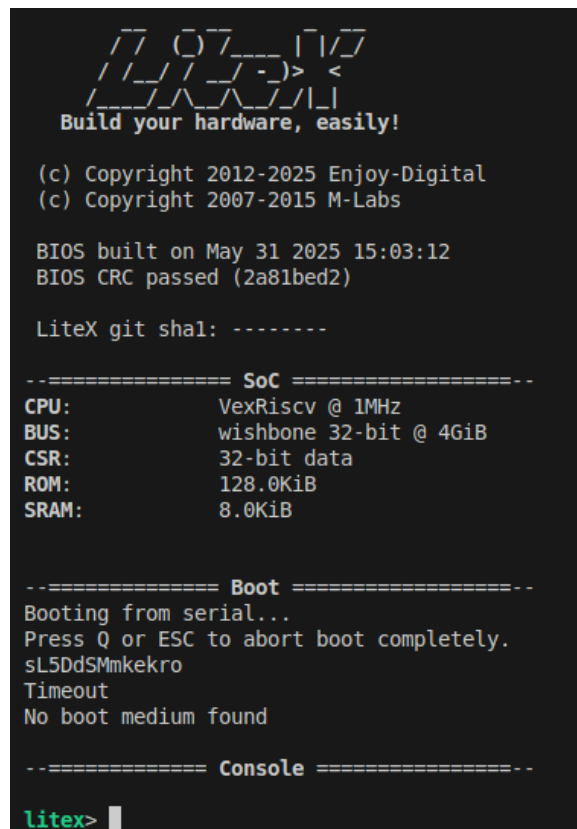


Figura 1: Início simulação com Litex.

O Litex disponibiliza um exemplo de demonstração para executar na CPU escolhida na simulação, através do comando:

```
litex_bare_metal_demo --build-path=build/sim/
```

Para executar o binário gerado na CPU pode-se utilizar o seguinte comando:

```
litex_sim --integrated-main-ram-size=0x10000 --cpu-type=vexriscv
--ram-init=demo.bin
```

A tela na Figura 2 mostra o sucesso da simulação com execução da demo. O exemplo possui alguns comandos disponíveis, como mostrado na Figura 2, e é possível escolher a opção para desenhar um donut, por exemplo, veja o resultado na Figura 3.

Passo 2 – Criação e Execução de um Novo Programa

Ferramentas Necessárias:

Ambiente configurado conforme o Passo 1.

Execução:

O arquivo `main.c` da demo foi substituída pelo seguinte programa:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libbase/uart.h>
#include <libbase/console.h>
#include <generated/csr.h>

//ajustar de acordo com teste
#define INI_REP          5000
#define QUANT_REP        50000

static void teste_exec(void)
{
    uint32_t i;
    puts("\nInicio teste\n");
    puts("\n3\n");
    for (i = 0; i < INI_REP; i++) {
        printf(".");
    }
    puts("\n2\n");
    for (i = 0; i < INI_REP; i++) {
        printf(".");
    }
    puts("\n1\n");
    for (i = 0; i < INI_REP; i++) {
        printf(".");
    }
    puts("\nComecou\n");
    for (i = 0; i < QUANT_REP; i++) {
```

```

        printf("%" PRIu32 " ", i);
    }
    puts("\nFim teste\n");
}

int main(void)
{
    uart_init();
    teste_exec();

    return 0;
}

```

Após a substituição, deve-se usar o comando para gerar o `demo.bin` novamente:

```
litex_bare_metal_demo --build-path=build/sim/
```

O objetivo geral do programa é executar uma contagem com sinalização de início para cronometrar o tempo de execução do programa. O tempo decorrido da execução do programa foi calculado seguindo a seguinte metodologia: Assim que a mensagem "Começou" é exibida, o cronômetro é iniciado. Após toda a execução, a mensagem "Fim teste" é mostrada e o cronômetro é parado. Foi utilizado o próprio cronômetro nativo do Linux. Após a medição, o programa apresentado demorou 42 segundos para ser executado no simulado com a CPU VexRiscv.

Ao executar o mesmo programa (com exceção da inicialização da UART e bibliotecas próprias do Litex) no sistema operacional Linux, o tempo de execução foi muito rápido, tornando impossível a medição. Então, todas as repetições foram ajustadas para x1000. Foi alterado apenas o início do código:

```

//ajustar de acordo com teste
#define INI_REP          5000*1000
#define QUANT_REP        50000*1000

```

O comando para execução do programa foi o seguinte:

```

gcc main.c -o main
./main

```

O tempo medido para a execução no Linux foi de 61 segundos com o loop ajustado para x1000. Na VexRiscv, o mesmo código levou 42 segundos, mas com o loop em sua forma original. Isso mostra uma diferença de 19 segundos, mesmo com o Linux executando uma carga 1000 vezes maior, evidenciando a grande diferença de desempenho entre um sistema operacional completo e a CPU sintetizada na FPGA.

Passo 3 – Troca do Processador Padrão por Outro Compatível e Comparação dos Resultados

Ferramentas Necessárias:

Ambiente configurado conforme os Passos 1 e 2.

Execução:

Para trocar de processador utilizando Litex, é muito simples, basta colocar o tipo de CPU no comando. A CPU escolhida é Coreblocks:

```
litex_sim --cpu-type=coreblocks
```

A tela resultante é mostrada na Figura 4, indicando a nova CPU escolhida.

[illegible]

Figura 4: Início simulação com CPU CoreBlocks.

Após a troca, é necessário gerar novamente o `.bin` do código C com o comando:

```
litex_bare_metal_demo --build-path=build/sim/
```

O código é enviado através do seguinte comando:

```
litex_sim --integrated-main-ram-size=0x10000 --cpu-type=coreblocks  
--ram-init=demo.bin
```

O tempo de execução utilizando o mesmo código apresentado no Passo 2 foi muito grande. Portanto, para esta CPU, foi utilizada uma grandeza de divisão por 25 nas repetições do código, a fim de facilitar a comparação. O código foi alterado apenas no início:

```
//ajustar de acordo com teste  
#define INI_REP          200  
#define QUANT_REP        2000
```

O tempo cronometrado, seguindo a metodologia já apresentada, foi de 17 segundos. A comparação entre VexRiscv, Coreblocks e Linux é resumida na Tabela 1:

Plataforma	Fator aplicado no loop	Tempo de execução real	Tempo estimado (equivalente a ×1)
VexRiscv	× 1	42 segundos	42 segundos
CoreBlocks	÷ 25	17 segundos	425 segundos
Linux	× 1000	61 segundos	0,061 segundos

Tabela 1: Comparação de Tempo de Execução (Simulador)

Passo 4 – Adição e Utilização de um Novo Periférico com TANG NANO 9K

Ferramentas Necessárias:

- OpenFPGALoader: Usado para comunicação com a placa Tang Nano 9K.
- Terminal Tabby: Usado para monitorar a USB do computador, encontrado em: <https://tabby.sh/?ref=learn.lushaylabs.com>

- Gowin IDE: Ferramenta de sintetização do fabricante da placa Tang Nano 9k. Pode ser baixada aqui: https://www.gowinsemi.com/en/support/download_eda

Ajuste de permissões para acessar a USB sem ser root:

```
curl -sSL  
https://raw.githubusercontent.com/lushaylabs/openfpgaloader-ubuntu  
fix/main/setup.sh | sh
```

Execução:

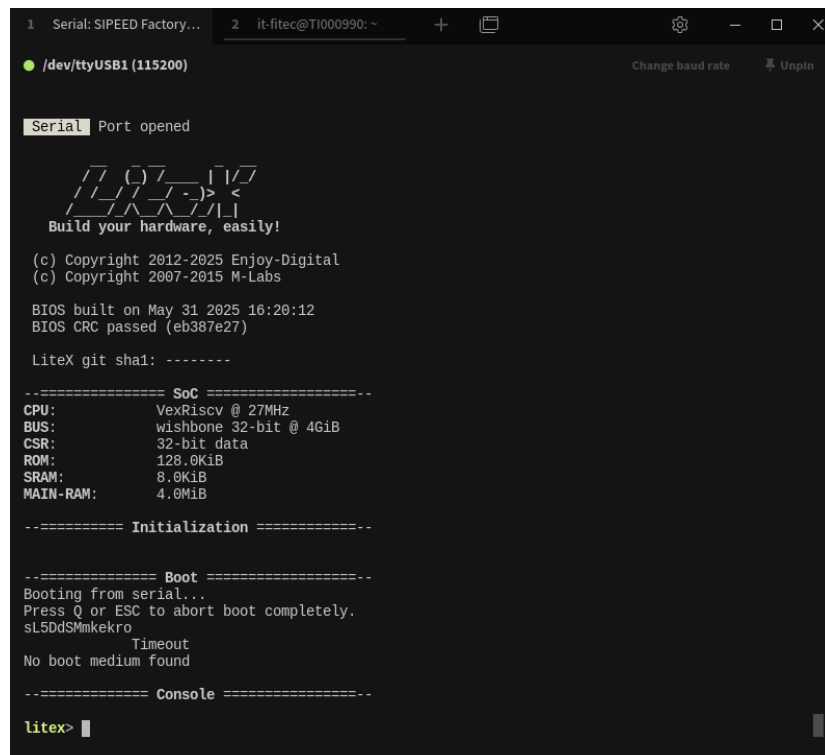
Dentro da pasta de instalação do Litex, existe uma pasta chamada `litex_board`, que é o ambiente que será utilizado a partir de agora no projeto. A integração do Litex com a sintetização da placa é realizada através de dois arquivos principais:

- `target/sipeed_tang_nano_9k.py`: Responsável por carregar os módulos que serão utilizados na CPU.
- `platforms/sipeed_tang_nano_9k.py`: Definição de hardware como pinagens da placa que serão utilizadas.

O seguinte comando pode ser utilizado para compilar o programa e sintetizar a placa:

```
python3 -m litex_boards.targets.sipeed_tang_nano_9k --build --load
```

A serial da placa pode ser monitorada através do USB com a ferramenta Tabby, após a execução do comando é mostrado na Figura 5 os dados do terminal.



```
1 Serial: SIPEED Factory... 2 it-fitec@T1000990: ~
/dev/ttyUSB1 (115200) Change baud rate Unpin

Serial Port opened

Build your hardware, easily!

(c) Copyright 2012-2025 Enjoy-Digital
(c) Copyright 2007-2015 M-Labs

BIOS built on May 31 2025 16:20:12
BIOS CRC passed (eb387e27)

LiteX git sha1: -----

----- SoC -----
CPU:      VexRiscv @ 27MHz
BUS:      wishbone 32-bit @ 4GiB
CSR:      32-bit data
ROM:      128.0KiB
SRAM:     8.0KiB
MAIN-RAM: 4.0MiB

----- Initialization -----

----- Boot -----
Booting from serial...
Press Q or ESC to abort boot completely.
sL5dSMmkekro
Timeout
No boot medium found

----- Console -----

litex>
```

Figura 5: Início simulação com Tang Nano 9K

Como periférico, foi adicionado o botão para início da simulação e acionamento/desacionamento de LED. As mudanças realizadas no código target base da placa foram a remoção da memória externa e inclusão do módulo de GPIO, segue mudanças abaixo:

```
from litex.soc.cores.gpio import *
...
class BaseSoC(SoCCore):
...
self.leds = GPIOOut(platform.request_all("user_led"))
self.gpio = GPIOIn(platform.request("user_btn", 1))
self.add_csr("gpio")
```

No arquivo `platform`, não foi necessária nenhuma mudança, pois os pinos correspondentes ao LED e botão já estavam configurados corretamente. O uso do timer para medição também foi adicionado para auxiliar a medição do tempo da execução do código. A versão da `main.c` ficou da seguinte maneira:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```

#include <irq.h>
#include <libbase/uart.h>
#include <libbase/console.h>
#include <generated/csr.h>

#define CLOCK_FREQUENCY 27000000 // 27 MHz
#define QUANT_REP      50000

volatile uint32_t botao_user = 0;

static void teste_exec(void)
{
    uint32_t i;
    uint32_t tempo_inicio, tempo_fim;
    uint64_t ciclos;
    int segundos;
    leds_out_write(0x00);
    puts("\nAperte o botão para iniciar o teste\n");
    botao_user = gpio_in_read();
    while(botao_user){
        botao_user = gpio_in_read();
    }
    leds_out_write(0x3F);
    puts("\nInicio teste\n");

    timer0_reload_write(0xFFFFFFFF);
    timer0_load_write(0xFFFFFFFF);
    timer0_en_write(1);

    timer0_update_value_write(1);
    tempo_inicio = timer0_value_read();

    for (i = 0; i < QUANT_REP; i++) {
        printf("%ld ", i);
    }
    timer0_update_value_write(1);
    tempo_fim = timer0_value_read();

    ciclos = (uint64_t)(tempo_inicio - tempo_fim);
    segundos = (ciclos / CLOCK_FREQUENCY);

    printf("\nTempo decorrido: %d segundos\n", segundos);
}

```

```

    leds_out_write(0x00);
    puts("\nFim teste\n");
}

int main(void)
{

#ifdef CONFIG_CPU_HAS_INTERRUPT
    irq_setmask(0);
    irq_setie(1);
#endif
    uart_init();
    teste_exec();

    return 0;
}

```

O programa pode ser enviado para a placa através dos comandos:

```

litex_bare_metal_demo --build-path=build/sipeed_tang_nano_9k
--mem=rom (Este comando deve ser executado após o build da CPU
escolhida)
python3 -m litex_boards.targets.sipeed_tang_nano_9k
--integrated-rom-init=demo.bin --cpu-type vexriscv --build --load

```

Após o envio do `.bin`, é necessário apertar o botão da placa para iniciar o teste, como mostra a Figura 6.

```
1 Serial: SIPEED Factory... + [icon] [icon] [icon]
● /dev/ttyUSB1 (115200) Change baud rate Unpin

  Build your hardware, easily!

(c) Copyright 2012-2025 Enjoy-Digital
(c) Copyright 2007-2015 M-Labs

BIOS built on May 31 2025 17:03:20
BIOS CRC passed (bc8f410a)

LiteX git sha1: -----

===== SoC =====
CPU:          VexRiscv @ 27MHz
BUS:          wishbone 32-bit @ 4GiB
CSR:          32-bit data
ROM:          128.0KiB
SRAM:         8.0KiB
MAIN-RAM:     4.0MiB

===== Initialization =====

===== Boot =====
Booting from serial...
Press Q or ESC to abort boot completely.
sL5DdSMmketro
Timeout
No boot medium found

===== Console =====

litex>
Aperte o botão para iniciar o teste
```

Figura 6: Início da execução do programa com Tang Nano 9k.

No fim da execução, apresenta o tempo gasto, que foi de 25 segundos, como mostrado na Figura 7.

```
1 Settings      2 Serial: SiP...  3 Serial: SiP...  +  [Icon]  [Gear]  -  [Box]  X

● /dev/ttyUSB1 (115200)  Change baud rate  Unpin

6 49637 49638 49639 49640 49641 49642 49643 49644 49645 49646 49647 49648 49649 49650 49651 4966
52 49653 49654 49655 49656 49657 49658 49659 49660 49661 49662 49663 49664 49665 49666 49667 49
668 49669 49670 49671 49672 49673 49674 49675 49676 49677 49678 49679 49680 49681 49682 49683 4
9684 49685 49686 49687 49688 49689 49690 49691 49692 49693 49694 49695 49696 49697 49698 49699
49700 49701 49702 49703 49704 49705 49706 49707 49708 49709 49710 49711 49712 49713 49714 49715
49716 49717 49718 49719 49720 49721 49722 49723 49724 49725 49726 49727 49728 49729 49730 4973
1 49732 49733 49734 49735 49736 49737 49738 49739 49740 49741 49742 49743 49744 49745 49746 497
47 49748 49749 49750 49751 49752 49753 49754 49755 49756 49757 49758 49759 49760 49761 49762 49
763 49764 49765 49766 49767 49768 49769 49770 49771 49772 49773 49774 49775 49776 49777 49778 4
9779 49780 49781 49782 49783 49784 49785 49786 49787 49788 49789 49790 49791 49792 49793 49794
49795 49796 49797 49798 49799 49800 49801 49802 49803 49804 49805 49806 49807 49808 49809 49810
49811 49812 49813 49814 49815 49816 49817 49818 49819 49820 49821 49822 49823 49824 49825 4982
6 49827 49828 49829 49830 49831 49832 49833 49834 49835 49836 49837 49838 49839 49840 49841 498
42 49843 49844 49845 49846 49847 49848 49849 49850 49851 49852 49853 49854 49855 49856 49857 49
858 49859 49860 49861 49862 49863 49864 49865 49866 49867 49868 49869 49870 49871 49872 49873 4
9874 49875 49876 49877 49878 49879 49880 49881 49882 49883 49884 49885 49886 49887 49888 49889
49890 49891 49892 49893 49894 49895 49896 49897 49898 49899 49900 49901 49902 49903 49904 49905
49906 49907 49908 49909 49910 49911 49912 49913 49914 49915 49916 49917 49918 49919 49920 4992
1 49922 49923 49924 49925 49926 49927 49928 49929 49930 49931 49932 49933 49934 49935 49936 499
37 49938 49939 49940 49941 49942 49943 49944 49945 49946 49947 49948 49949 49950 49951 49952 49
953 49954 49955 49956 49957 49958 49959 49960 49961 49962 49963 49964 49965 49966 49967 49968 4
9969 49970 49971 49972 49973 49974 49975 49976 49977 49978 49979 49980 49981 49982 49983 49984
49985 49986 49987 49988 49989 49990 49991 49992 49993 49994 49995 49996 49997 49998 49999
Tempo decorrido: 25 segundos

Fim teste
```

Figura 7: Fim execução Tang Nano 9k.

Entre as CPUs disponíveis no LiteX, apenas a CPU Minerva apresenta sucesso na compatibilidade com a placa Tang Nano 9K, utilizando o mesmo processo de geração, sendo alterando apenas o nome da CPU nos comandos. Esse resultado indica que apenas a VexRiscv e Minerva são atualmente as únicas, dentre as testadas, que funcionam corretamente com a combinação LiteX + ferramentas da Gowin + Tang Nano 9K, sem a necessidade de modificações adicionais no projeto. Segue lista sequencial dos comandos utilizados:

```
python3 -m litex_boards.targets.sipeed_tang_nano_9k --cpu-type
minerva --build
litex_bare_metal_demo --build-path=build/sipeed_tang_nano_9k
--mem=rom
python3 -m litex_boards.targets.sipeed_tang_nano_9k
--integrated-rom-init=demo.bin --cpu-type minerva --build --load
```

O tempo de execução com a CPU Minerva foi igual ao VexRiscv na Tang Nano 9K, igual a 25 segundos.

Conclusão final:

A Tabela 2 a seguir mostra a comparação entre os testes realizados no simulador e no hardware físico com a placa Tang Nano 9K, utilizando o mesmo programa básico em C. Percebe-se que a execução no Linux é muito mais rápida em comparação com a CPU sintetizada neste projeto, e que a CPU VexRiscv se mostra mais eficiente do que a CoreBlocks. Além disso, a mesma CPU VexRiscv apresentou desempenho superior quando executada na placa Tang Nano 9K em relação ao simulador.

Plataforma	CPU	Fator aplicado no loop	Tempo de execução real	Tempo estimado (equivalente a ×1)
Simulador litex	VexRiscv	× 1	42 segundos	42 segundos
Tang Nano 9K	VexRiscv	× 1	25 segundos	25 segundos
Simulador litex	CoreBlocks	÷ 25	17 segundos	425 segundos
Linux	x86_64	× 1000	61 segundos	0,061 segundos

Tabela 2: Comparação de Tempo de Execução (Simulador x Tang Nano 9K)

Após as modificações e inclusão dos periféricos, o programa foi executado com a CPU VexRiscv e Minerva na Tang Nano 9K, e o tempo de execução foi o mesmo em ambas as CPUs.

Apesar das dificuldades iniciais na preparação do ambiente, a documentação do LiteX se mostrou eficaz para a continuidade do projeto. A plataforma demonstrou ser bastante eficiente para personalização no estilo "*plug and play*". Como visto no projeto, é possível trocar a CPU com apenas um comando.

O código *demo* utilizado como base foi disponibilizado pela própria LiteX. Para destacar as diferenças em relação ao código oficial e as modificações feitas para este projeto, foi criado um repositório no GitHub disponível em: <https://github.com/Cleidiana/Litex-TangNano9k>. Neste repositório:

- A pasta `litex_sim` contém os arquivos utilizados nos testes com o simulador do LiteX;
- A pasta `litex_boards` contém os arquivos para execução na Tang Nano 9K;
- A pasta `teste_linux` traz o código usado para a execução em ambiente Linux.